

Reviewer Pack — Additive Energy Inverse Proportionality to Karchmer-Wigderson...

Entry 521a5bf7dd7d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 13:55:48 UTC

Additive Energy Inverse Proportionality to Karchmer-Wigderson Communication Complexity

Entry ID: 521a5bf7dd7d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 13:55:48 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics **Field B** (complexity object): Karchmer-Wigderson game

Statement:

For every boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the additive energy $E(f)$ of its truth table satisfies $E(f) \leq 2^n / (C(f) \cdot \log n)$, where $C(f)$ is the communication complexity of the Karchmer-Wigderson game for f .

Rationale (proposer's reasoning):

Additive energy captures the number of additive quadruples in the truth table, reflecting algebraic structure that could enable efficient communication protocols. The inverse proportionality suggests that higher structure (lower energy) correlates with lower communication complexity.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ad70b2c7b12f3d66

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def compute_additive_energy(cnf):
        n = len(cnf[0])
        energy = 0
        for a in range(2**n):
            for b in range(a+1, 2**n):
                if sum([abs(int(x) - int(y)) for x, y in zip(bin(a)[2:].zfill(n), bin(b)[2:].zfill(n))]) > 0:
                    energy += 1
        return energy

    def compute_communication_complexity(cnf):
        # Simple deterministic protocol: each variable is queried once
        return len(cnf[0])

    def generate_random_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(random.randint(1, n))]
            cnf.append(clause)
        return cnf

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(5): # Ensure at least 5 instances per size
            cnf = generate_random_cnf(n, random.randint(1, n))
            energy = compute_additive_energy(cnf)
            C = compute_communication_complexity(cnf)
            if C == 0: # Avoid division by zero
                continue
            results.append({
                "n": n,
                "energy": energy,
```

```

        "C": C,
        "log_n": math.log(n, 2),
        "metric_value": energy * C * math.log(n, 2)
    })

if not results:
    return {
        "metric_name": "additive_energy",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "no_instances_generated"
    }

metric_values = [r["metric_value"] for r in results]
instances_tested = len(results)
conjecture_holds = all(v <= 2**n / (C * math.log(n)) for n, C, v in zip([r["n"] for r in results]))

return {
    "metric_name": "additive_energy",
    "metric_value": sum(metric_values) / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "energy * C * log n > 2^n"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"energy * C * log n > 2^n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_results")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	63450
2	preregistration	ollama_remote	qwen3:8b	0	0	27826
3	novelty	ollama_remote	qwen3:8b	0	0	24247
4	novelty	ollama_remote	qwen3:8b	0	0	17739
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13209
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11432
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16334
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10712
9	judge	ollama_remote	qwen3:8b	0	0	20912

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 205861 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/521a5bf7dd7d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/521a5bf7dd7d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/521a5bf7dd7d.tar.gz (if generated)